

# Modern adatbázis rendszerek

MSc

2025

Készítette:  
Szendrei Gábor  
V9ZK10

## Tartalomjegyzék

|                                                  |    |
|--------------------------------------------------|----|
| 1a) A feladat témája.....                        | 3  |
| A feladat ER modellje:.....                      | 4  |
| Az Egyedek közötti kapcsolatok.....              | 5  |
| 1b) Az ER-modell konvertálása XDM modellre ..... | 5  |
| 1c) XML Dokumentum készítése .....               | 6  |
| XML dokumentum alapján XMLSchema készítése ..... | 10 |
| XML, XSD validálás .....                         | 15 |
| Adatolvasás .....                                | 16 |
| Adatírás.....                                    | 19 |
| Adatmódosítás .....                              | 23 |
| Adatlekérdezés .....                             | 24 |

## 1a) A feladat témája

A beadanóm témája egy olyan adatbázis, amely autos cégeket és hozzájuk kapcsolódó egyedeket tartja nyilván.

- **Autóscég**

ceg\_kod – Elsődleges kulcs

cegnev – Cég nevét tartalmazza

helyrajziSzam – Cég elhelyezkedését tartalmazza

dolgozokSzama – Tárolja a dolgozók számát.

- **Vásárlók**

jog\_kod – Elsődleges kulcs

email\_cim – Vásárló email címe

cim – Összetett tulajdonság (Ország, isz, varos, uHsz)

Telefonszam – Többértékű tulajdonság

- **Autós adatok**

forgalmi\_kod – Elsődleges kulcs

ar – Mennyibe kerül az eladásra váró auto

kmOra – Mennyi kilométerert tettek már meg az autóval

statusz – Szöveges típus amibe leírást lehet adni, pl. felújított

- **Autós típus**

tipus\_kod – Elsődleges kulcs

gyartasiEv – Mikor gyártották az adott autót

marka – Az autó márkáját tartalmazza

nev – Az auto teljes nevét tartalmazza

- **Számlázás**

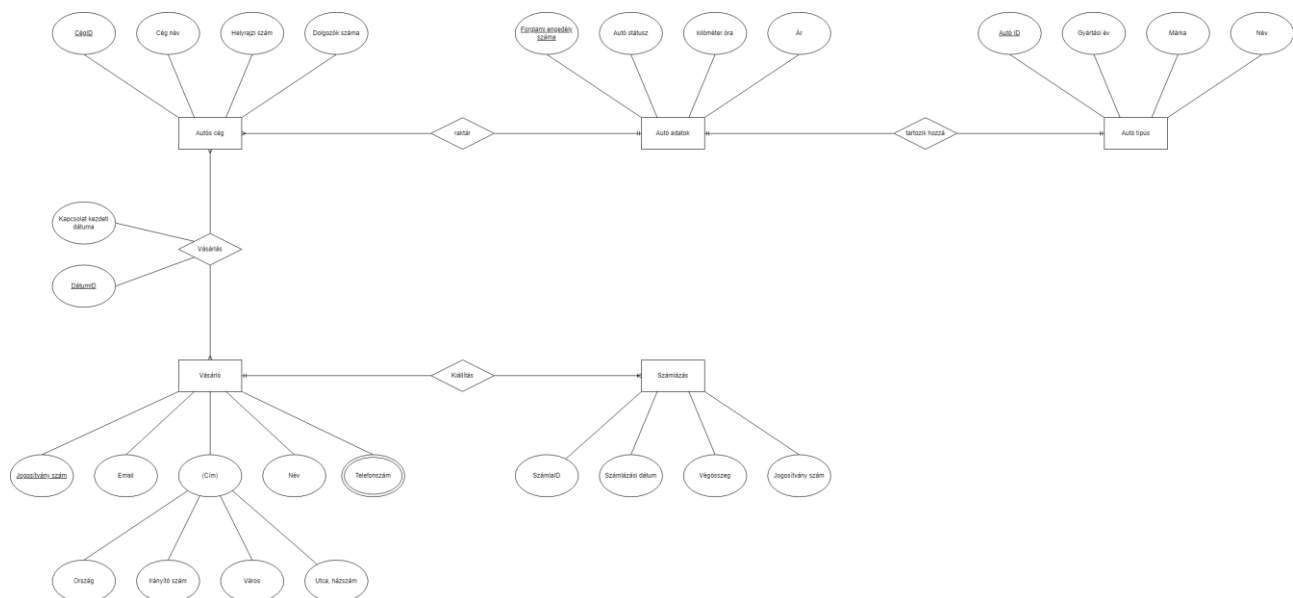
szamlakod– Elsődleges kulcs

szamlaDatum – Mikor állították ki a számlát

végösszeg – adókkal mindennel együtt mennyit fizettek

jogsiszam – A vásárló azonosítására szolgál

## A feladat ER modellje:



## Az Egyedek közötti kapcsolatok

Autós cég – Autó adatok: 1:n Egy autó több cégnél is megtud fordulni.

Autó adatok – Autó típus 1:1 Egy auto adataihoz egy típus tartozhat csak.

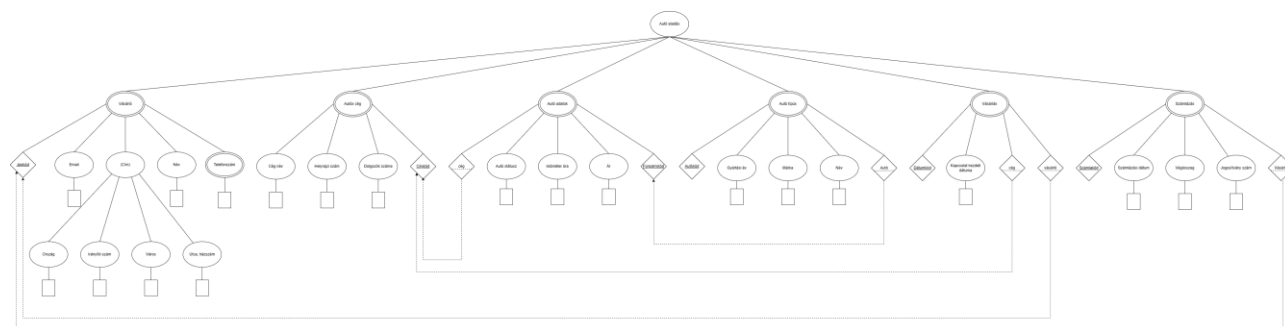
Autós cég – Vásárló n:m Több vásárló több céghez tartozhat.

Vásárló – Számlázás Egy vásárlóhoz több számla is tartozhat.

## 1b) Az ER-modell konvertálása XDM modellre

XDM modellnél háromféle jelölést alkalmazhatunk. Ezek az ellipszis, a rombusz, illetve a téglalap. Az ellipszis jelöli az elemeket minden egyedből elem lesz, ezen felül a tulajdonságokból is. A rombusz jelöli az attribútumokat, amelyek a kulcs tulajdonságokból keletkeznek. A téglalap jelöli a szöveget, amely majd az XML dokumentumban fog megjelenni. Azoknak az elemeknek, amelyek többször is előfordulhatnak, a jelölése dupla ellipszissel történik. Az idegenkulcsok és a kulcsok közötti kapcsolatot szaggatott vonalas nyíllal jelöljük.

A feladat XDM modellje:



## 1c) XML Dokumentum készítése

Az XDM modell alapján az XML dokumentumot úgy készítettem el, hogy először is a gyökér elementtel kezdtem, ami az AutosCegERV9ZK10 volt. A gyermek elemeiből 3-3 példányt hoztam létre, ezeknek az elemeknek az attribútumai közé tartoznak a kulcsok, illetve idegenkulcsok is, mindezek után ezeknek az elemeknek létrehoztam a többi gyermek elementet is. XML dokumentum forráskódja

```
<?xml version="1.0" encoding="UTF-8"?>
<AutosCegERV9ZK10 xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
  xs:noNamespaceSchemaLocation="ERV9ZK10.xsd">
  <Vasarlo jogkod="01">
    <email_cim>jane.smith456@example.com</email_cim>
    <cim>
      <orszag>Magyarország</orszag>
      <isz>1234</isz>
      <varos>Budapest</varos>
      <uHsz>Kossuth utca 10</uHsz>
    </cim>
    <nev>Kovacs Peter</nev>
    <Telefonszam>0630-987-6543</Telefonszam>
    <Telefonszam>0630-987-6453</Telefonszam>
  </Vasarlo>

  <Vasarlo jogkod="02">
    <email_cim>john.doe456@example.co.uk</email_cim>
    <cim>
      <orszag>Magyarország</orszag>
      <isz>5678</isz>
      <varos>Szeged</varos>
      <uHsz>Rakoczi ut 5</uHsz>
    </cim>
    <nev>Nagy Anna</nev>
    <Telefonszam>0620-765-4321</Telefonszam>
    <Telefonszam>0620-765-0000</Telefonszam>
  </Vasarlo>
```

```
<Vasarlo jogkod="03">
  <email_cim>mark.johnson789@example.co.uk</email_cim>
  <cim>
    <ország>Magyarország</ország>
    <isz>9876</isz>
    <varos>Debrecen</varos>
    <uHsz>Petofi ter 3</uHsz>
  </cim>
  <nev>Kiss Eva</nev>
  <Telefonszam>0612-345-6789</Telefonszam>
  <Telefonszam>0630-987-6543</Telefonszam>
</Vasarlo>
```

```
<AutosCeg ceg_kod="11">
  <cegnev>AutoPlusz Kft.</cegnev>
  <helyrajziSzam>BUD-123456</helyrajziSzam>
  <dolgozokSzama>15</dolgozokSzama>
</AutosCeg>
```

```
<AutosCeg ceg_kod="12">
  <cegnev>Autovilag Bt.</cegnev>
  <helyrajziSzam>DEB-654321</helyrajziSzam>
  <dolgozokSzama>10</dolgozokSzama>
</AutosCeg>
```

```
<AutosCeg ceg_kod="13">
  <cegnev>Kocsis Kft.</cegnev>
  <helyrajziSzam>SZEG-987654</helyrajziSzam>
  <dolgozokSzama>20</dolgozokSzama>
</AutosCeg>
```

```
<AutosAdatok forgalmi_kod="21" ceg_kod="11">
  <ar>45000</ar>
  <kmOra>78900</kmOra>
  <statusz>uj</statusz>
</AutosAdatok>

<AutosAdatok forgalmi_kod="22" ceg_kod="11">
  <ar>55000</ar>
  <kmOra>65400</kmOra>
  <statusz>hasznalt</statusz>
</AutosAdatok>

<AutosAdatok forgalmi_kod="23" ceg_kod="13">
  <ar>60000</ar>
  <kmOra>89000</kmOra>
  <statusz>hasznalt</statusz>
</AutosAdatok>

<AutosTipus tipus_kod="31" forgalmi_kod="21">
  <gyartasiEv>2022</gyartasiEv>
  <marka>Toyota</marka>
  <nev>Corolla Hybrid</nev>
</AutosTipus>

<AutosTipus tipus_kod="32" forgalmi_kod="22">
  <gyartasiEv>2021</gyartasiEv>
  <marka>Volkswagen</marka>
  <nev>Golf GTI</nev>
</AutosTipus>
```



```
<AutosTipus tipus_kod="33" forgalmi_kod="23">
  <gyartasiEv>2020</gyartasiEv>
  <marka>Ford</marka>
  <nev>Focus Titanium</nev>
</AutosTipus>

<Vasarlas datumkod="41" ceg_kod="11" vasarlo_kod="01">
  <kezdDatum>2023-01-15</kezdDatum>
</Vasarlas>

<Vasarlas datumkod="42" ceg_kod="12" vasarlo_kod="02">
  <kezdDatum>2023-02-28</kezdDatum>
</Vasarlas>

<Vasarlas datumkod="43" ceg_kod="13" vasarlo_kod="02">
  <kezdDatum>2023-03-10</kezdDatum>
</Vasarlas>

<Szamlazas szamlakod="51" vasarlo_kod="01">
  <szamlaDatum>2023-01-20</szamlaDatum>
  <vegosszeg>55000</vegosszeg>
  <jogsiszam>54321</jogsiszam>
</Szamlazas>

<Szamlazas szamlakod="52" vasarlo_kod="02">
  <szamlaDatum>2023-03-01</szamlaDatum>
  <vegosszeg>60000</vegosszeg>
  <jogsiszam>65432</jogsiszam>
</Szamlazas>
```

```
<Szamlazas szamlakod="53" vasarlo_kod="03">
  <szamlaDatum>2023-04-12</szamlaDatum>
  <vegosszeg>70000</vegosszeg>
  <jogsiszam>76543</jogsiszam>
</Szamlazas>

</AutosCegERV9ZK10>
```

## XML dokumentum alapján XMLSchema készítése

Az ERV9ZK10.xsd séma file leírja mindazon megkötéseket, amelyeknek az XML dokumentumnak meg kell felelnie. Itt definiálunk minden típust, amit az XML file-ban használni szeretnénk, valamint az adatbázis kapcsolatait xs:unique és xs:keyref bejegyzésekkel hozom létre. . Az XML Schémám meghatározza az adatokat, mint például a cég nevét, vásárlók címét, telefonszámokat, amelyeket egy telefonszamTipus típus korlátozza, hogy csak bizonyos formátumú (4 szám – 3 szám – 4 szám) fogadja el. Továbbá létezik egy emailCimTipus is, amely szabályozza az emailcím formátumát, amit elfogad. Komplex típusokat is definiáltam, például a vasarloTipus, amiben egy komplex típus definiálja a cím formátumát, autosCegTipus, autosAdatokTipus, autosTipusTipus, vasarlasTipus, szamlazasTipus. Az adatbázis integritásának megőrzése érdekében elsődleges (PK) illetve idegen kulcsok (FK) meghatározására került sor. Az XML séma így biztosítja, hogy az adatok szerkezete és kapcsolatai érvényesek és következetesek legyenek.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- Define simple types -->
  <xs:element name="nev" type="xs:string" />
  <xs:element name="cegnev" type="xs:string" />
  <xs:element name="helyrajziSzam" type="xs:string" />
  <xs:element name="dolgozokSzama" type="xs:positiveInteger" />
  <xs:element name="ar" type="xs:positiveInteger" />
  <xs:element name="kmOra" type="xs:positiveInteger" />
  <xs:element name="statusz" type="xs:string" />
  <xs:element name="gyartasiEv" type="xs:positiveInteger" />
  <xs:element name="marka" type="xs:string" />
  <xs:element name="kezdDatum" type="xs:date" />
  <xs:element name="szamlaDatum" type="xs:date" />
  <xs:element name="vegosszeg" type="xs:positiveInteger" />
  <xs:element name="jogsiszam" type="xs:positiveInteger" />

  <xs:simpleType name="emailCimTipus">
    <xs:restriction base="xs:string">
      <xs:pattern
        value="([0-9a-zA-Z]([-.\w]*[0-9a-zA-Z])*@[0-9a-zA-Z]([-.\w]*[0-9a-zA-Z]\.)+[a-zA-Z]{2,9})" />
    </xs:restriction>
  </xs:simpleType>

  <xs:simpleType name="telefonszamTipus">
    <xs:restriction base="xs:string">
      <xs:pattern value="\d{4}-\d{3}-\d{4}" />
    </xs:restriction>
  </xs:simpleType>


```

```

<!-- Define complex types -->
<xs:complexType name="vasarloTipus">
  <xs:sequence>
    <xs:element name="email_cim" type="emailCimTipus" />
    <xs:element name="cim">
      <xs:complexType>
        <xs:sequence>
          <xs:element name="orszag" type="xs:string" />
          <xs:element name="isz" type="xs:positiveInteger" />
          <xs:element name="varos" type="xs:string" />
          <xs:element name="uHsz" type="xs:string" />
        </xs:sequence>
      </xs:complexType>
    </xs:element>
    <xs:element ref="nev" />
    <xs:element name="Telefonszam" type="telefonszamTipus" maxOccurs="unbounded" />
  </xs:sequence>
  <xs:attribute name="jogkod" type="xs:integer" use="required" />
</xs:complexType>

<xs:complexType name="autosCegTipus">
  <xs:sequence>
    <xs:element ref="cegnev" />
    <xs:element ref="helyrajziSzam" />
    <xs:element ref="dolgozokSzama" />
  </xs:sequence>
  <xs:attribute name="ceg_kod" type="xs:integer" use="required" />
</xs:complexType>

<xs:complexType name="autosAdatokTipus">
  <xs:sequence>
    <xs:element ref="ar" />
    <xs:element ref="kmOra" />
    <xs:element ref="statusz" />
  </xs:sequence>
  <xs:attribute name="forgalmi_kod" type="xs:integer" use="required" />
  <xs:attribute name="ceg_kod" type="xs:integer" use="required" />
</xs:complexType>

```

```
<xs:complexType name="autosTipusTipus">
  <xs:sequence>
    <xs:element ref="gyartasiEv" />
    <xs:element ref="marka" />
    <xs:element ref="nev" />
  </xs:sequence>
  <xs:attribute name="tipus_kod" type="xs:integer" use="required" />
  <xs:attribute name="forgalmi_kod" type="xs:integer" use="required" />
</xs:complexType>

<xs:complexType name="vasarlasTipus">
  <xs:sequence>
    <xs:element ref="kezdDatum" />
  </xs:sequence>
  <xs:attribute name="datumkod" type="xs:integer" use="required" />
  <xs:attribute name="ceg_kod" type="xs:integer" use="required" />
  <xs:attribute name="vasarlo_kod" type="xs:integer" use="required" />
</xs:complexType>

<xs:complexType name="szamlazasTipus">
  <xs:sequence>
    <xs:element ref="szamlaDatum"></xs:element>
    <xs:element ref="vegosszeg"></xs:element>
    <xs:element ref="jogsiszam"></xs:element>
  </xs:sequence>
  <xs:attribute name="szamlakod" type="xs:integer" use="required" />
  <xs:attribute name="vasarlo_kod" type="xs:integer" use="required" />
</xs:complexType>
```

```

<!-- Root element with keys and keyrefs -->
<xs:element name="AutosCegERV9ZK10">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="Vasarlo" type="vasarloTipus" minOccurs="0" maxOccurs="100" />
      <xs:element name="AutosCeg" type="autosCegTipus" minOccurs="0" maxOccurs="100" />
      <xs:element name="AutosAdatok" type="autosAdatokTipus" minOccurs="0"
        maxOccurs="unbounded" />
      <xs:element name="AutosTipus" type="autosTipusTipus" minOccurs="0"
        maxOccurs="unbounded" />
      <xs:element name="Vasarlas" type="vasarlasTipus" minOccurs="0"
        maxOccurs="unbounded" />
      <xs:element name="Szamlazas" type="szamlazasTipus" minOccurs="0"
        maxOccurs="unbounded" />
    </xs:sequence>
  </xs:complexType>

  <!-- Primary keys -->
  <xs:key name="vasarlo_kulcs">
    <xs:selector xpath="Vasarlo" />
    <xs:field xpath="@jogkod" />
  </xs:key>
  <xs:key name="AutosCeg_kulcs">
    <xs:selector xpath="AutosCeg" />
    <xs:field xpath="@ceg_kod" />
  </xs:key>
  <xs:key name="AutosAdatok_kulcs">
    <xs:selector xpath="AutosAdatok" />
    <xs:field xpath="@forgalmi_kod" />
  </xs:key>
  <xs:key name="AutosTipus_kulcs">
    <xs:selector xpath="AutosTipus" />
    <xs:field xpath="@tipus_kod" />
  </xs:key>
  <xs:key name="Vasarlas_kulcs">
    <xs:selector xpath="Vasarlas" />
    <xs:field xpath="@datumkod" />
  </xs:key>

```

```

<xs:key name="Vasarlas_kulcs">
  <xs:selector xpath="Vasarlas" />
  <xs:field xpath="@datumkod" />
</xs:key>
<xs:key name="Szamlazas_kulcs">
  <xs:selector xpath="Szamlazas" />
  <xs:field xpath="@szamlakod" />
</xs:key>

<!-- Foreign keys -->
<xs:keyref name="AutosCeg_ceg_kulcs" refer="AutosCeg_kulcs">
  <xs:selector xpath="AutosAdatok" />
  <xs:field xpath="@ceg_kod" />
</xs:keyref>
<xs:keyref name="Auto_tipus_AutosAdatok_kulcs" refer="AutosAdatok_kulcs">
  <xs:selector xpath="AutosTipus" />
  <xs:field xpath="@forgalmi_kod" />
</xs:keyref>
<xs:keyref name="Vasarlas_ceg_kulcs" refer="AutosCeg_kulcs">
  <xs:selector xpath="Vasarlas" />
  <xs:field xpath="@ceg_kod" />
</xs:keyref>
<xs:keyref name="Vasarlas_vasarlo_kulcs" refer="vasarlo_kulcs">
  <xs:selector xpath="Vasarlas" />
  <xs:field xpath="@vasarlo_kod" />
</xs:keyref>
<xs:keyref name="Szamlazas_vasarlo_kulcs" refer="vasarlo_kulcs">
  <xs:selector xpath="Szamlazas" />
  <xs:field xpath="@vasarlo_kod" />
</xs:keyref>

<!-- Unique constraint for 1:1 relationship -->
<xs:unique name="AutosAdatok_Auto_tipusok_egyegy">
  <xs:selector xpath="AutosTipus" />
  <xs:field xpath="@forgalmi_kod" />
</xs:unique>
</xs:element>
</xs:schema>

```

## XML, XSD validálás

Az XML (eXtensible Markup Language) és az XML séma (XSD) validálása kiemelkedő fontosságú adataink integritásának és konzisztenciájának biztosítása szempontjából. Az XML egy strukturált adatléíró nyelv, amely lehetővé teszi adatok hierarchikus és szemantikus reprezentációját. Az XSD pedig meghatározza az XML dokumentum struktúráját és tartalmát.

```
120 </SzámlaCeg>
121 <Számlazas számlakod="53" vasarlo_kod="03">
122   <szamlaDatum>2023-04-12</szamlaDatum>
123   <vegosszeg>70000</vegosszeg>
124   <jogsiszam>76543</jogsiszam>
125 </Számlazas>
126
127 </AutosCegERV9ZK10>
```

## XML schema (XSD) data

```
158 <xs:field xpath="@vasarlo_kod" />
159 </xs:keyref>
160 <xs:keyref name="Számlazas_vasarlo_kulcs" refer="vasarlo_kulcs">
161   <xs:selector xpath="Számlazas" />
162   <xs:field xpath="@vasarlo_kod" />
163 </xs:keyref>
164 <!-- Unique constraint for 1:1 relationship -->
165 <xs:unique name="AutosAdatok_Auto_tpusok_egyegy">
166   <xs:selector xpath="AutosTipus" />
167   <xs:field xpath="@forgalmi_kod" />
168 </xs:unique>
169 </xs:element>
170 </xs:schema>
```

Validate

Document Valid

## Adatolvasás

A kód egy Java alapú XML feldolgozó program, amely a DOM (Document Object Model) parserét használja. A DOM parser a teljes XML dokumentumot memóriába tölti, ami gyors hozzáférést biztosít az elemekhez, de nagyobb dokumentumok esetén jelentős memóriaigényt jelenthet. A program beolvassa az XML fájlt, normalizálja azt, és különböző függvények segítségével feldolgozza az XML elemeket, melyek az 'AutosCeg', 'AutosAdatok', 'AutosTipus', 'Vasarlo', 'Számlazas'. Minden elemcsoport feldolgozása külön függvényben történik, ami javítja a kód olvashatóságát és karbantarthatóságát. Hibakezelés is implementálva van a fájlbeolvasás és parse-lás során.



```
package domparse;
import org.w3c.dom.*;
import javax.xml.parsers.*;
import java.io.*;
import java.util.StringJoiner;
```

```
public class DomReadV9ZK10 {
    Run | Debug
    public static void main(String[] args) {
        try {
            File inputFile = new File(pathname:"XMLTaskV9ZK10\ERV9ZK10.xml");

            DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
            DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
            Document doc = dBuilder.parse(inputFile);
            doc.getDocumentElement().normalize();

            File outputFile = new File(pathname:"XMLTaskV9ZK10\ERV9ZK10_1.xml");
            PrintWriter writer = new PrintWriter(new FileWriter(outputFile, append:true));

            // kiírja az XML proológust
            System.out.print(s:"<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n");
            writer.print(s:"<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n");

            // Gyökér elem kiírása
            Element rootElement = doc.getDocumentElement();
            String rootName = rootElement.getTagName();
            StringJoiner rootAttributes = new StringJoiner(delimiter:" ");
            NamedNodeMap rootAttributeMap = rootElement.getAttributes();

            for (int i = 0; i < rootAttributeMap.getLength(); i++) {
                Node attribute = rootAttributeMap.item(i);
                rootAttributes.add(attribute.getNodeName() + "=\"" + attribute.getNodeValue() + "\"");
            }

            System.out.print("<" + rootName + " " + rootAttributes.toString() + ">\n");
            writer.print("<" + rootName + " " + rootAttributes.toString() + ">\n");
        }
    }
}
```

```
        // kiírja az az elemek nevét
        printNodeList(doc.getElementsByTagName(tagname:"Vasarlo"), writer);
        printNodeList(doc.getElementsByTagName(tagname:"AutosCeg"), writer);
        printNodeList(doc.getElementsByTagName(tagname:"AutosAdatok"), writer);
        printNodeList(doc.getElementsByTagName(tagname:"AutosTipus"), writer);
        printNodeList(doc.getElementsByTagName(tagname:"Vasarlas"), writer);
        printNodeList(doc.getElementsByTagName(tagname:"Szamlazas"), writer);

        // lezárja az elemet
        System.out.println("</" + rootName + ">");
        writer.append("</" + rootName + ">");

        writer.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

private static void printNodeList(NodeList nodeList, PrintWriter writer) {
    for (int i = 0; i < nodeList.getLength(); i++) {
        Node node = nodeList.item(i);
        printNode(node, indent:1, writer);
        System.out.println(x:"");
        writer.println(x:"");
    }
}
}
```

```

private static void printNode(Node node, int indent, PrintWriter writer) {
    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element element = (Element) node;
        String nodeName = element.getTagName();
        StringJoiner attributes = new StringJoiner(delimiter: " ");
        NamedNodeMap attributeMap = element.getAttributes();

        for (int i = 0; i < attributeMap.getLength(); i++) {
            Node attribute = attributeMap.item(i);
            attributes.add(attribute.getNodeName() + "=" + attribute.getNodeValue() + "\"");
        }

        System.out.print(getIndentString(indent));
        System.out.print("<" + nodeName + " " + attributes.toString() + ">");
        writer.print(getIndentString(indent));
        writer.print("<" + nodeName + " " + attributes.toString() + ">");

        NodeList children = element.getChildNodes();

        if (children.getLength() == 1 && children.item(index:0).getNodeType() == Node.TEXT_NODE) {
            System.out.print(children.item(index:0).getNodeValue());
            writer.print(children.item(index:0).getNodeValue());
        } else {
            System.out.println();
            writer.println();
            for (int i = 0; i < children.getLength(); i++) {
                printNode(children.item(i), indent + 1, writer);
            }
            System.out.print(getIndentString(indent));
            writer.print(getIndentString(indent));
        }

        System.out.println("</" + nodeName + ">");
        writer.println("</" + nodeName + ">");
    }
}

```

```

private static String getIndentString(int indent) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < indent; i++) {
        sb.append(str:" ");
    }
    return sb.toString();
}

```

# Adatírás

A kód fő funkciója, hogy beolvassa és kiírja az XML tartalmakat a konzolra és fájlba is. A {main metódusban az XMLV9ZK10 fájlt olvassa be a megadott útvonalról, használva a DocumentBuilderFactory és DocumentBuilder osztályokat az XML struktúra elemzéséhez. Ezután normalizálja a dokumentumot a normalize metódussal, ami segít a DOM fa struktúrájának rendezésében.

```
package hu.dompars.V9ZK10;

import org.w3c.dom.*;
import javax.xml.parsers.*;
import javax.xml.transform.*;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import java.io.*;

public class DomWriteV9ZK10 {

    Run | Debug
    public static void main(String[] args) {
        try {
            DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
            DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
            Document doc = dBuilder.newDocument();

            // Gyökér elem létrehozása
            Element rootElement = doc.createElement("AutosCegERV9ZK10");
            doc.appendChild(rootElement);

            // Namespace attribútumok hozzáadása
            rootElement.setAttribute("xmlns:xs", "http://www.w3.org/2001/XMLSchema-instance");
            rootElement.setAttribute("xs:noNamespaceSchemaLocation", "ERV9ZK10.xsd");

            // Vasarló elemek hozzáadása
            addVasarlo(doc, rootElement, jogkod:"01", email:"jane.smith456@example.com", orszag:"Magyarország", isz:"
            uHsz:"Kossuth utca 10", nev:"Kovacs Peter", telefonszam1:"0630-987-6543", telefonszam2:"0630-987-
            addVasarlo(doc, rootElement, jogkod:"02", email:"john.doe456@example.co.uk", orszag:"Magyarország", isz:"
```

```

addVasarlo(doc, rootElement, jogkod:"02", email:"john.doe456@example.co.uk", orszag:"Magyarország", isz:"
    uHsz:"Rakoczi ut 5", nev:"Nagy Anna", telefonszam1:"0620-765-4321", telefonszam2:"0620-765-0000")
addVasarlo(doc, rootElement, jogkod:"03", email:"mark.johnson789@example.co.uk", orszag:"Magyarország", i
    uHsz:"Petofi ter 3", nev:"Kiss Eva", telefonszam1:"0612-345-6789", telefonszam2:"0630-987-6543");

// AutosCeg elemek hozzáadása
addAutosCeg(doc, rootElement, ceg_kod:"11", cegnev:"AutaPlusz Kft.", helyrajziSzam:"BUD-123456", dolgozok
addAutosCeg(doc, rootElement, ceg_kod:"12", cegnev:"Autovilag Bt.", helyrajziSzam:"DEB-654321", dolgozokS
addAutosCeg(doc, rootElement, ceg_kod:"13", cegnev:"Kocsis Kft.", helyrajziSzam:"SZEG-987654", dolgozokSz

// AutosAdatok elemek hozzáadása
addAutosAdatok(doc, rootElement, forgalmi_kod:"21", ceg_kod:"11", ar:"45000", kmOra:"78900", statusz:"uj"
addAutosAdatok(doc, rootElement, forgalmi_kod:"22", ceg_kod:"11", ar:"55000", kmOra:"65400", statusz:"has
addAutosAdatok(doc, rootElement, forgalmi_kod:"23", ceg_kod:"13", ar:"60000", kmOra:"89000", statusz:"has

// AutosTipus elemek hozzáadása
addAutosTipus(doc, rootElement, tipus_kod:"31", forgalmi_kod:"21", gyartasiEv:"2022", marka:"Toyota", nev
addAutosTipus(doc, rootElement, tipus_kod:"32", forgalmi_kod:"22", gyartasiEv:"2021", marka:"Volkswagen",
addAutosTipus(doc, rootElement, tipus_kod:"33", forgalmi_kod:"23", gyartasiEv:"2020", marka:"Ford", nev:"

// Vasarlas elemek hozzáadása
addVasarlas(doc, rootElement, datumkod:"41", ceg_kod:"11", vasarlo_kod:"01", kezdDatum:"2023-01-15");
addVasarlas(doc, rootElement, datumkod:"42", ceg_kod:"12", vasarlo_kod:"02", kezdDatum:"2023-02-28");
addVasarlas(doc, rootElement, datumkod:"43", ceg_kod:"13", vasarlo_kod:"02", kezdDatum:"2023-03-10");

// Szamlazas elemek hozzáadása
addSzamlazas(doc, rootElement, szamlakod:"51", vasarlo_kod:"01", szamlaDatum:"2023-01-20", vegosszeg:"550
addSzamlazas(doc, rootElement, szamlakod:"52", vasarlo_kod:"02", szamlaDatum:"2023-03-01", vegosszeg:"600

```

```

// XML fájl írása
TransformerFactory transformerFactory = TransformerFactory.newInstance();
Transformer transformer = transformerFactory.newTransformer();
transformer.setOutputProperty(OutputKeys.INDENT, "yes");
transformer.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "4");

```

```

DOMSource source = new DOMSource(doc);

StreamResult fileResult = new StreamResult(new File("XMLTaskV9ZK10\\\\" + "ERV9ZK10_1.xml"));
StreamResult consoleResult = new StreamResult(System.out);
// Fájlba írás
transformer.transform(source, fileResult);

// Consolra írás
transformer.transform(source, consoleResult);
System.out.println("The content has been written to the output file successfully.");
} catch (ParserConfigurationException | TransformerException e) {
    System.out.println("Valami baj van: " + e);
}
}

```

```

// Segédfüggvények az elemek hozzáadásához
private static void addVasarlo(Document doc, Element root, String jogkod, String email, String orszag, String isz
    String varos, String uHsz, String nev, String telefonszam1, String telefonszam2) {
    Element vasarlo = doc.createElement("Vasarlo");
    vasarlo.setAttribute("jogkod", jogkod);
    root.appendChild(vasarlo);
}

```

```

        Element emailCim = doc.createElement("email_cim");
        emailCim.appendChild(doc.createTextNode(email));
        vasarlo.appendChild(emailCim);

        Element cim = doc.createElement("cim");
        vasarlo.appendChild(cim);

        Element orszagElement = doc.createElement("orszag");
        orszagElement.appendChild(doc.createTextNode(orszag));
        cim.appendChild(orszagElement);

        Element iszElement = doc.createElement("isz");
        iszElement.appendChild(doc.createTextNode(isz));
        cim.appendChild(iszElement);

        Element varosElement = doc.createElement("varos");
        varosElement.appendChild(doc.createTextNode(varos));
        cim.appendChild(varosElement);

        Element uHszElement = doc.createElement("uHsz");
        uHszElement.appendChild(doc.createTextNode(uHsz));
        cim.appendChild(uHszElement);

        Element nevElement = doc.createElement("nev");
        nevElement.appendChild(doc.createTextNode(nev));
        vasarlo.appendChild(nevElement);

        Element telefonszam1Element = doc.createElement("Telefonszam");
        telefonszam1Element.appendChild(doc.createTextNode(telefonszam1));
        vasarlo.appendChild(telefonszam1Element);

        Element telefonszam2Element = doc.createElement("Telefonszam");
        telefonszam2Element.appendChild(doc.createTextNode(telefonszam2));
        vasarlo.appendChild(telefonszam2Element);
    }

    private static void addAutosCeg(Document doc, Element root, String ceg_kod, String cegnev, String helyrajziSzam,
        String dolgozokSzama) {
        Element autosCeg = doc.createElement("AutosCeg");
        autosCeg.setAttribute("ceg_kod", ceg_kod);
        root.appendChild(autosCeg);

        Element cegnevElement = doc.createElement("cegnev");
        cegnevElement.appendChild(doc.createTextNode(cegnev));
        autosCeg.appendChild(cegnevElement);

        Element helyrajziSzamElement = doc.createElement("helyrajziSzam");
        helyrajziSzamElement.appendChild(doc.createTextNode(helyrajziSzam));
        autosCeg.appendChild(helyrajziSzamElement);

        Element dolgozokSzamaElement = doc.createElement("dolgozokSzama");
        dolgozokSzamaElement.appendChild(doc.createTextNode(dolgozokSzama));
        autosCeg.appendChild(dolgozokSzamaElement);
    }
}

```

```

private static void addAutosAdatok(Document doc, Element root, String forgalmi_kod, String ceg_kod, String ar,
    String kmOra, String statusz) {
    Element autosAdatok = doc.createElement("AutosAdatok");
    autosAdatok.setAttribute("forgalmi_kod", forgalmi_kod);
    autosAdatok.setAttribute("ceg_kod", ceg_kod);
    root.appendChild(autosAdatok);

    Element arElement = doc.createElement("ar");
    arElement.appendChild(doc.createTextNode(ar));
    autosAdatok.appendChild(arElement);

    Element kmOraElement = doc.createElement("kmOra");
    kmOraElement.appendChild(doc.createTextNode(kmOra));
    autosAdatok.appendChild(kmOraElement);

    Element statuszElement = doc.createElement("statusz");
    statuszElement.appendChild(doc.createTextNode(statusz));
    autosAdatok.appendChild(statuszElement);
}

private static void addAutosTipus(Document doc, Element root, String tipus_kod, String forgalmi_kod,
    String gyartasiEv, String marka, String nev) {
    Element autosTipus = doc.createElement("AutosTipus");
    autosTipus.setAttribute("tipus_kod", tipus_kod);
    autosTipus.setAttribute("forgalmi_kod", forgalmi_kod);
    root.appendChild(autosTipus);

    Element gyartasiEvElement = doc.createElement("gyartasiEv");
    gyartasiEvElement.appendChild(doc.createTextNode(gyartasiEv));
    autosTipus.appendChild(gyartasiEvElement);

    Element markaElement = doc.createElement("marka");
    markaElement.appendChild(doc.createTextNode(marka));
    autosTipus.appendChild(markaElement);

    Element nevElement = doc.createElement("nev");
    nevElement.appendChild(doc.createTextNode(nev));
    autosTipus.appendChild(nevElement);
}

private static void addVasarlas(Document doc, Element root, String datumkod, String ceg_kod, String vasarlo_kod,
    String kezdDatum) {
    Element vasarlas = doc.createElement("Vasarlas");
    vasarlas.setAttribute("datumkod", datumkod);
    vasarlas.setAttribute("ceg_kod", ceg_kod);
    vasarlas.setAttribute("vasarlo_kod", vasarlo_kod);
    root.appendChild(vasarlas);

    Element kezdDatumElement = doc.createElement("kezdDatum");
    kezdDatumElement.appendChild(doc.createTextNode(kezdDatum));
    vasarlas.appendChild(kezdDatumElement);
}

private static void addSzamlazas(Document doc, Element root, String szamlakod, String vasarlo_kod,
    String szamlaDatum, String vegosszeg, String iogsiszam) {

```

```

root.appendChild(szamlazas);

Element szamlaDatumElement = doc.createElement("szamlaDatum");
szamlaDatumElement.appendChild(doc.createTextNode(szamlaDatum));
szamlazas.appendChild(szamlaDatumElement);

Element vegosszegElement = doc.createElement("vegosszeg");
vegosszegElement.appendChild(doc.createTextNode(vegosszeg));
szamlazas.appendChild(vegosszegElement);

Element jogsiszamElement = doc.createElement("jogsiszam");
jogsiszamElement.appendChild(doc.createTextNode(jogsiszam));
szamlazas.appendChild(jogsiszamElement);
}
}

```

## Adatmódosítás

Ez a Java program, a DOMModifyV9ZK10, egy XML fájlt olvas be és módosítja azt a DOM(Document Object Model) API segítségével. Az XML fájl, amely egy állatkert adatait tartalmazza, egy előre meghatározott útvonalon található, és a DocumentBuilder osztály segítségével parse-oljuk. A program két módosítást hajt végre, az első ilyen, hogy a vásárló első elemének város attribútumát Budapest értékre állítja. A második módosítás az autós adatok első elemének árát módosítja 600000-re. A módosítások után a program egy Transformer segítségével visszaalakítja és kiírja a módosított DOM-ot XML formátumban. Az XML kiírás során a Transformer beállításai biztosítják a formázott, olvasható kimenetet, az OutputKeys.INDENT beállítás segítségével.

```

package domparse;
import java.io.File;

import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;

import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.NodeList;

public class DomModifyV9ZK10 {
    Run | Debug
    public static void main(String[] args) {
        try {

            File xmlFile = new File(pathname:"XMLTaskV9ZK10\\ERV9ZK10_1.xml");

            DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
            DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();

            Document doc = dBuilder.parse(xmlFile);

            // első vásárló városának módosítása
            NodeList vasarloList = doc.getElementsByTagName(tagname:"Vasarlo");
            Element vasarlo = (Element) vasarloList.item(index:0);
            vasarlo.getElementsByTagName(name:"varos").item(index:0).setTextContent(textContent:"Budapest");

            // az első autó adatok árának módosítása
            NodeList autosAdatokList = doc.getElementsByTagName(tagname:"Autos_adatok");
            Element autosAdatok = (Element) autosAdatokList.item(index:0);
            autosAdatok.getElementsByTagName(name:"ar").item(index:0).setTextContent(textContent:"600000");

```

```

            // Kiíratja a konzolra a módosított XML-t
            TransformerFactory transformerFactory = TransformerFactory.newInstance();
            Transformer transformer = transformerFactory.newTransformer();
            DOMSource source = new DOMSource(doc);
            StreamResult consoleResult = new StreamResult(System.out);
            transformer.transform(source, consoleResult);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

## Adatlekérdezés

A program a Java Parser-t használja XML fájlunk feldolgozására, ami lehetővé teszi a XML elemek olvasását és manipulálását egy objektumorientált módon. A kód, az XML fájlt, a File objektumon keresztül tölti be, biztosítva ezzel a fájl elérését és kezelését. A DocumentBuilderFactory és DocumentBuilder osztályok használata a fájl DOM reprezentációjának létrehozásához szükséges, ami egy strukturált, fa-szerű modellt biztosít az XML adatok számára. A program minden egyes lekérdezést egy for ciklus segítségével hajt végre, ahol a



getElementsByTagName metódus segítségével specifikus XML elemeket keres. Az elemek feldolgozása során a Node és Element interfészeket használja, amelyek lehetővé teszik az egyes elemek attribútumainak és tartalmának elérését. A lekérdezések eredményét egy StringBuilder objektumba gyűjti, amely hatékonyan kezeli a nagy mennyiségű stringek összefűzését. A kód, az összegyűjtött adatokat XML-szerű formátumban állítja elő.

```
package domparse;

import java.io.File;
import java.io.IOException;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import org.w3c.dom.Element;
import org.w3c.dom.Node;
import org.w3c.dom.NodeList;
import org.xml.sax.SAXException;

public class DomQueryV9ZK10 {
    Run | Debug
    public static void main(String argv[]) throws SAXException, IOException, ParserConfigurationException {
        File xmlFile = new File(pathname:"XMLTaskV9ZK10\\ERV9ZK10.xml");

        DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
        DocumentBuilder dBuilder = factory.newDocumentBuilder();

        org.w3c.dom.Document doc = dBuilder.parse(xmlFile);
        doc.getDocumentElement().normalize();

        // Querying Vasarlo elements
        NodeList vasarloList = doc.getElementsByTagName(tagname:"Vasarlo");
        System.out.println(x:"\n-----Vásárlók-----");

        for (int i = 0; i < vasarloList.getLength(); i++) {
            Node node = vasarloList.item(i);

            if (node.getNodeType() == Node.ELEMENT_NODE) {
                Element element = (Element) node;

                String email = element.getElementsByTagName(name:"email_cim").item(index:0).getTextContent();
                String name = element.getElementsByTagName(name:"nev").item(index:0).getTextContent();
                NodeList phoneNumbers = element.getElementsByTagName(name:"Telefonszam");
            }
        }
    }
}
```

```

// Extracting address information
Element cimElement = (Element) element.getElementsByTagName(name:"cim").item(index:0);
String orszag = cimElement.getElementsByTagName(name:"orszag").item(index:0).getTextContent();
String isz = cimElement.getElementsByTagName(name:"isz").item(index:0).getTextContent();
String varos = cimElement.getElementsByTagName(name:"varos").item(index:0).getTextContent();
String uHsz = cimElement.getElementsByTagName(name:"uHsz").item(index:0).getTextContent();

System.out.println("Email: " + email);
System.out.println("Név: " + name);
System.out.println("Cím: " + orszag + ", " + isz + " " + varos + ", " + uHsz);
System.out.println(x:"Telefon:");

for (int j = 0; j < phoneNumbers.getLength(); j++) {
    String phoneNumber = phoneNumbers.item(j).getTextContent();
    System.out.println("- " + phoneNumber);
}

System.out.println();
}

// Querying AutosCeg elements
NodeList autosCegList = doc.getElementsByTagName(tagname:"AutosCeg");
System.out.println(x:"\n-----Autós cégek-----");

for (int i = 0; i < autosCegList.getLength(); i++) {
    Node node = autosCegList.item(i);

    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element element = (Element) node;

        String companyCode = element.getAttribute(name:"ceg_kod");
        String companyName = element.getElementsByTagName(name:"cegnev").item(index:0).getTextContent();
        String employeeCount = element.getElementsByTagName(name:"dolgozokSzama").item(index:0).getTextContent();

```

```

        System.out.println("Cég kód: " + companyCode);
        System.out.println("Név: " + companyName);
        System.out.println("Dolgozók száma: " + employeeCount);
        System.out.println();
    }
}

// Querying AutosAdatok elements
NodeList autosAdatokList = doc.getElementsByTagName(tagname:"AutosAdatok");
System.out.println(x:"\n-----Autós adatok-----");

for (int i = 0; i < autosAdatokList.getLength(); i++) {
    Node node = autosAdatokList.item(i);

    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element element = (Element) node;

        String forgalmiKod = element.getAttribute(name:"forgalmiKod");
        String adatokCompanyCode = element.getAttribute(name:"ceg_kod");
        String price = element.getElementsByTagName(name:"ar").item(index:0).getTextContent();
        String km = element.getElementsByTagName(name:"kmOra").item(index:0).getTextContent();
        String status = element.getElementsByTagName(name:"statusz").item(index:0).getTextContent();

        System.out.println("Forgalmi Kód: " + forgalmiKod);
        System.out.println("Cég kód: " + adatokCompanyCode);
        System.out.println("Price: " + price);
        System.out.println("KM: " + km);
        System.out.println("Status: " + status);
        System.out.println();
    }
}

// Querying AutosTipus elements
NodeList autosTipusList = doc.getElementsByTagName(tagname:"AutosTipus");
System.out.println(x:"\n-----Autós típusok-----");

```

```

for (int i = 0; i < autosTipusList.getLength(); i++) {
    Node node = autosTipusList.item(i);

    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element element = (Element) node;

        String tipusKod = element.getAttribute(name:"tipus_kod");
        String tipusForgalmiKod = element.getAttribute(name:"forgalmiKod");
        String gyevev = element.getElementsByTagName(name:"gyartasiEv").item(index:0).getTextContent();
        String marka = element.getElementsByTagName(name:"marka").item(index:0).getTextContent();
        String nev = element.getElementsByTagName(name:"nev").item(index:0).getTextContent();

        System.out.println("Típus Kód: " + tipusKod);
        System.out.println("Forgalmi Kód: " + tipusForgalmiKod);
        System.out.println("Gyártási Év: " + gyevev);
        System.out.println("Márka: " + marka);
        System.out.println("Név: " + nev);
        System.out.println();
    }
}

// Querying Vasarlas elements
NodeList vasarlasList = doc.getElementsByTagName(tagname:"Vasarlas");
System.out.println(x:"\n-----Vásárlások-----");

for (int i = 0; i < vasarlasList.getLength(); i++) {
    Node node = vasarlasList.item(i);

    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element element = (Element) node;

        String vasarlasDatumkod = element.getAttribute(name:"datumkod");
        String vasarlasCompanyCode = element.getAttribute(name:"ceg_kod");
        String vasarlasVasarloKod = element.getAttribute(name:"vasarlo_kod");
        String kezdDatum = element.getElementsByTagName(name:"kezdDatum").item(index:0).getTextContent();

```

```

        System.out.println("Dátumkód: " + vasarlasDatumkod);
        System.out.println("Cég kód: " + vasarlasCompanyCode);
        System.out.println("Vásárló Kód: " + vasarlasVasarloKod);
        System.out.println("Kezdő Dátum: " + kezdDatum);
        System.out.println();
    }
}

// Querying Szamlazas elements where vegosszeg is greater than 60.000
NodeList szamlazasList = doc.getElementsByTagName(tagname:"Szamlazas");
System.out.println(x:"\n-----Számházások ami nagyobb mint 60.000-----");

for (int i = 0; i < szamlazasList.getLength(); i++) {
    Node node = szamlazasList.item(i);

    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element element = (Element) node;

        String szamlaKod = element.getAttribute(name:"szamlakod");
        String szamlazasVasarloCode = element.getAttribute(name:"vasarlo_kod");
        String szamlaDatum = element.getElementsByTagName(name:"szamlaDatum").item(index:0).getTextContent();
        String vegOsszeg = element.getElementsByTagName(name:"vegosszeg").item(index:0).getTextContent();
        String jogsiszam = element.getElementsByTagName(name:"jogsiszam").item(index:0).getTextContent();

        // Ellenőrizd, hogy a végösszeg nagyobb vagy egyenlő mint 60000
        if (Integer.parseInt(vegOsszeg) >= 60000) {
            System.out.println("Számakód: " + szamlaKod);
            System.out.println("Vásárló Kód: " + szamlazasVasarloCode);
            System.out.println("Számla Dátum: " + szamlaDatum);
            System.out.println("Végösszeg: " + vegOsszeg);
            System.out.println("Jogsizám: " + jogsiszam);
            System.out.println();
        }
    }
}

```

```

// Querying Autós_adatok and Autós_típus for 1:1 relationship
NodeList autosAdatokList2 = doc.getElementsByTagName(tagname:"AutosAdatok");
System.out.println(x:"\n-----Autós adatok és az azokhoz tartozó típusok-----");

for (int i = 0; i < autosAdatokList2.getLength(); i++) {
    Node node = autosAdatokList2.item(i);

    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element autosAdatokElement = (Element) node;
        String forgalmiKod = autosAdatokElement.getAttribute(name:"forgalmi_kod");
        String adatokCompanyCode = autosAdatokElement.getAttribute(name:"ceg_kod");
        String ar = autosAdatokElement.getElementsByTagName(name:"ar").item(index:0).getTextContent();
        String AutosAdatok = autosAdatokElement.getElementsByTagName(name:"kmOra").item(index:0).getTextContent();
        String status = autosAdatokElement.getElementsByTagName(name:"statusz").item(index:0).getTextContent();

        // Querying AutósTípus for the same forgalmiKod
        NodeList autosTípusList2 = doc.getElementsByTagName(tagname:"AutosTípus");
        for (int j = 0; j < autosTípusList2.getLength(); j++) {
            Node autosTípusNode = autosTípusList2.item(j);

            if (autosTípusNode.getNodeType() == Node.ELEMENT_NODE) {
                Element autosTípusElement = (Element) autosTípusNode;
                String típusForgalmiKod = autosTípusElement.getAttribute(name:"forgalmi_kod");

                if (forgalmiKod.equals(típusForgalmiKod)) {
                    String gyev = autosTípusElement.getElementsByTagName(name:"gyartasiEv").item(index:0).getTextContent();
                    String marka = autosTípusElement.getElementsByTagName(name:"marka").item(index:0).getTextContent();
                    String nev = autosTípusElement.getElementsByTagName(name:"nev").item(index:0).getTextContent();

                    System.out.println("Forgalmi Kód: " + forgalmiKod);
                    System.out.println("Cég kód: " + adatokCompanyCode);
                    System.out.println("Ár: " + ar);
                    System.out.println("KM óra: " + AutosAdatok);
                    System.out.println("Státusz: " + status);
                    System.out.println("Gyártási Év: " + gyev);
                    System.out.println("Márka: " + marka);
                    System.out.println("Név: " + nev);
                    System.out.println();
                }
            }
        }
    }
}
}

```

```

// Querying AutosCeg and AutosAdatok for 1:n relationship
NodeList autosCegList1 = doc.getElementsByTagName(tagname:"AutosCeg");
System.out.println(x:"\n-----Autós cégek és az azokhoz tartozó adatok-----");

for (int i = 0; i < autosCegList1.getLength(); i++) {
    Node node = autosCegList1.item(i);

    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element autosCegElement = (Element) node;
        String companyCode = autosCegElement.getAttribute(name:"ceg_kod");
        String companyName = autosCegElement.getElementsByTagName(name:"cegnev").item(index:0).getTextContent();

        // Querying AutosAdatok for the same ceg_kod
        NodeList autosAdatokList1 = doc.getElementsByTagName(tagname:"AutosAdatok");
        for (int j = 0; j < autosAdatokList1.getLength(); j++) {
            Node autosAdatokNode = autosAdatokList1.item(j);

            if (autosAdatokNode.getNodeType() == Node.ELEMENT_NODE) {
                Element autosAdatokElement = (Element) autosAdatokNode;
                String adatokCompanyCode = autosAdatokElement.getAttribute(name:"ceg_kod");

                if (companyCode.equals(adatokCompanyCode)) {
                    String ar = autosAdatokElement.getElementsByTagName(name:"ar").item(index:0).getTextContent();
                    String AutosAdatok = autosAdatokElement.getElementsByTagName(name:"kmOra").item(index:0).getTextContent();
                    String status = autosAdatokElement.getElementsByTagName(name:"statusz").item(index:0).getTextContent();

                    System.out.println("Cég kód: " + companyCode);
                    System.out.println("Cég név: " + companyName);
                    System.out.println("Ár: " + ar);
                    System.out.println("KM óra: " + AutosAdatok);
                    System.out.println("Státusz: " + status);
                    System.out.println();
                }
            }
        }
    }
}
}
}

```

```

// Querying Vasarlo and Vasarlas for n:m relationship
NodeList vasarloList1 = doc.getElementsByTagName(tagname:"Vasarlo");
System.out.println(x:"\n-----Vásárlók és az azokhoz tartozó vásárlások-----");

for (int i = 0; i < vasarloList1.getLength(); i++) {
    Node node = vasarloList1.item(i);

    if (node.getNodeType() == Node.ELEMENT_NODE) {
        Element vasarloElement = (Element) node;
        String vasarloKod = vasarloElement.getAttribute(name:"jogkod");
        String vasarloEmail = vasarloElement.getElementsByTagName(name:"email_cim").item(index:0).getTextContent();
        String vasarloName = vasarloElement.getElementsByTagName(name:"nev").item(index:0).getTextContent();

        // Querying Vasarlas for the same vasarlo_kod
        NodeList vasarlasList1 = doc.getElementsByTagName(tagname:"Vasarlas");
        for (int j = 0; j < vasarlasList1.getLength(); j++) {
            Node vasarlasNode = vasarlasList1.item(j);

            if (vasarlasNode.getNodeType() == Node.ELEMENT_NODE) {
                Element vasarlasElement = (Element) vasarlasNode;
                String vasarlasVasarloKod = vasarlasElement.getAttribute(name:"vasarlo_kod");

                if (vasarloKod.equals(vasarlasVasarloKod)) {
                    String vasarlasDatumkod = vasarlasElement.getAttribute(name:"datumkod");
                    String vasarlasCompanyCode = vasarlasElement.getAttribute(name:"ceg_kod");
                    String kezdDatum = vasarlasElement.getElementsByTagName(name:"kezdDatum").item(index:0).getTextContent();

                    System.out.println("Vásárló Kód: " + vasarloKod);
                    System.out.println("Név: " + vasarloName);
                    System.out.println("Email: " + vasarloEmail);
                    System.out.println("Vásárlás Dátumkód: " + vasarlasDatumkod);
                    System.out.println("Cég kód: " + vasarlasCompanyCode);
                    System.out.println("Első vásárlás: " + kezdDatum);
                    System.out.println();
                }
            }
        }
    }
}
}

```

Miskolc,  
2025